

AI 에이전트 프로덕션 설계 패턴

관측성, 실패 처리, 운영 가능한 신뢰성 확보를 위한 실용적
가이드

AI 에이전트 프로덕션 설계 패턴

관측성, 실패 처리, 운영 가능한 신뢰성 확보를 위한 실용적 가이드

ThakiCloud (Hyojung Han)

Contents

1	에이전트를 믿을 수 없는 세 가지 이유	1
2	관측성 설계: 에이전트의 행동을 읽을 수 있게 만들기	4
3	실패 처리 아키텍처: 에이전트가 붕괴하지 않는 구조	7
4	운영 가능한 신뢰성: 실제로 동작하는 시스템으로 만드는 법	10

1 에이전트를 믿을 수 없는 세 가지 이유

LLM 기반 에이전트가 프로덕션 환경에서 예상과 다르게 동작하는 데는 크게 세 가지 이유가 있다. 비결정적 출력과 조용히 전파되는 실패, 상태 관리의 부재다. 이 장에서는 세 문제가 실제로 어떤 형태로 나타나는지 살펴보고, 전통적인 소프트웨어 디버깅 방법이 에이전트에는 왜 통하지 않는지 설명한다.

1.1 비결정적 출력이라는 본질적 문제

에이전트의 첫 번째 특성은 동일한 입력에도 매번 다른 출력을 낼 수 있다는 것이다. Temperature를 0으로 고정해도 모델의 토큰 샘플링 방식과 컨텍스트 윈도우 내 주의 분포 변화 때문에 결과는 미세하게 달라진다. 이 특성은 단위 테스트로 검증하는 전형적인 소프트웨어와는 근본적으로 다르다.

예를 들어 파일명을 정규화하는 간단한 도구를 생각해보자. 전통적 소프트웨어라면 입력 “myFile.txt”에 대해 출력 “myfile.txt”이 항상 보장된다. 그러나 LLM 에이전트에게 같은 작업을 시키면 때로는 “myfile.txt”를 반환하고 때로는 “my_file.txt”를 반환하며 심지어 “myFile.txt” 그대로를 돌려줄 수도 있다. 여기에 temperature 등의 요인이 더해지면 출력의 분산은 더 커진다.

이것이 중요한 이유는 개발 환경에서 완벽해 보인 에이전트가 운영 환경에서는 예기치 못한 출력을 낼 수 있다는 데 있다. 프로덕션 시스템에서 이 문제를 관리하려면 출력의 허용 범위를 명시적으로 정의하고 그 범위를 벗어나는 출력을 자동으로 검출해 거부하는 검증 계층을 두어야 한다.

1.2 조용히 전파되는 실패

두 번째 문제는 에이전트 내부에서 생긴 실패가 이후 단계에 조용히 영향을 미친다는 점이다. 전통적 소프트웨어에서는 예외가 발생하면 스택 트레이스가 출력되어 즉시 확인할 수 있다. 그러나 에이전트에서는 중간 단계의 도구 호출이 실패해도 모델이 그 사실을 명시적으로 언급하지 않고 다음 단계를 그대로 진행하기도 한다.

구체적인 예를 하나 보자. 에이전트가 세 단계로 구성된 작업을 수행한다고 하자. 첫 번째 단계에서 데이터베이스 연결이 실패하고 두 번째 단계에서는 그 결과 일부만 반환되며 세 번째 단계에서 모델이 이 불완전한 결과를 바탕으로 최종 답변을 만들어낸다. 최종 출력만 보면 왜 답변이 잘못됐는지 알 수 없다. 데이터베이스 연결 실패라는 근본 원인이 중간 단계에 숨어 있기 때문이다.

이 문제의 본질은 에이전트의 추론 체인이 하나의 블랙 박스로 동작한다는 데 있다. 각 단계의 입력과 출력이 명시적으로 기록되지 않으면 실패의 근본 원인을 역추적하기가 사실상 불가능하다.

1.3 상태 관리의 부재

세 번째 문제는 에이전트가 호출 사이에 상태를 유지하지 않는다는 점이다. 사용자가 첫 질문을 던지면 에이전트는 적절히 답한다. 그런데 두 번째 질문을 하면 이전 대화의 맥락을 완전히 잃어버린 것처럼 동작하기도 한다. 이것은 에이전트 아키텍처 자체의 제약에서 비롯된 현상이다.

컨텍스트 윈도우가 유한하기 때문에 대화가 길어지면 이전 내용이 자연스럽게 떨어져 나간다. 모델은 유실된 정보를 추론으로 채우려 하는데 이때 사실에 기반하지 않은 내용을 만들어내기도 한다. 이것이 환각 hallucination의 한 형태이며, 에이전트가 프로덕션에서 실패하는 가장 흔한 이유 중 하나다.

이 세 가지 문제는 서로 독립적이지 않고 복잡하게 얽혀 있다. 비결정적 출력이 실패의 근본 원인을 감추고 상태 관리 부재가 그 실패를 증폭시키며 조용히 전파된 실패는 최종 사용자에게 전혀 예측할 수 없는 형태로 나타난다. 이 장에서 설명하는 패턴들은 이 문제들을 각각 따로 해결하는 것이 아니라 세 가지를 한꺼번에 다룰 수 있는 통합적 접근을 제공한다.

2 관측성 설계: 에이전트의 행동을 읽을 수 있게 만들기

에이전트가 프로덕션에서 실패했을 때 무엇이 잘못되었는지 파악하려면 에이전트의 내부 동작을 외부에서 관찰할 수 있어야 한다. 이것이 관측성 설계의 핵심이다. 이 장에서는 Span 기반 추적, 도구 호출의 분리 기록, 성능 게이트라는 세 가지 기법을 다룬다.

2.1 Span 기반 추적으로 호출 체인 시각화하기

Span은 작업의 시작과 끝 사이 시간 구간을 나타내는 개념이다. 에이전트의 각 단계마다 고유한 Span을 생성하고 그 안에 하위 작업을 자식으로 연결하면 호출 체인의 계층 구조가 완성된다.

예를 들면 이렇다. 에이전트가 사용자의 질의를 받으면 먼저 최상위 Span을 생성한다. 그 안에서 검색 도구를 호출하면 검색 Span을 자식으로 생성하고, 검색 결과를 바탕으로 응답을 생성하면 생성 Span을 또 다른 자식으로 연결한다. 검색에 200밀리초, 응답 생성에 3초가 걸렸다면 각 Span의 duration 필드에 그 값이 자동으로 기록된다.

이 구조가 유용한 이유는 실패가 발생한 Span만 필터링하면 문제의 위치를 바로 알 수 있다는 데 있다. 응답 생성 Span이 빨간색으로 표시되면 모델 추론 단계에 문제가 생겼다는 뜻이고, 검색 Span이 빨간색이면 검색 API 연결이 끊어졌다는 뜻이다. 전체 로그를 뒤지지 않아도 Span 트리만 보면 root cause에 도달할 수 있다.

실제 구현에서는 OpenTelemetry 스펙을 따르는 편이 낫다. 트레이스 ID, Span ID, parent ID가 포함된 표준 스키마를 쓰면 Jaeger, Tempo, Honeycomb 등 다양한 백엔드와 호환된다.

2.2 도구 호출 결과와 모델 추론 분리 기록하기

에이전트 관측성에서 가장 중요한 구분은 도구 호출의 입력·출력과 모델의 추론 내용을 나눠서 기록하는 것이다. 이 둘을 섞으면 로그가 빠르게 부풀고, 정작 필요한 정보를 찾기 어려워진다.

도구 호출 로그는 구조화된 형식으로 기록한다. 도구 이름, 호출 시점, 입력 매개변수, 출력 결과, 소요 시간을 각 도구 호출마다 남긴다. 출력 결과가 크면 전체 대신 처음 500자만 기록하고, 이후 조회가 필요할 때는 object storage에 FULL_LOG를 저장해 URI만 로그에 남기는 방식도 효과적이다.

모델 추론 로그는 다르게 다룬다. 모델이 생성한 사고 내용을 전부 저장하면 토큰 비용이 너무 크다. 대신 모델이 도구 호출을 결정한 직접적 이유, 즉 가장 중요했던 상위 3개 고려 요인만 요약해서 남긴다. 이렇게 하면 모델의 판단 과정을 대략 파악하면서도 로그 볼륨은 합리적으로 유지할 수 있다.

이 둘을 잇는 것이 추적 ID다. 에이전트를 호출할 때마다 고유한 추적 ID를 만들고, 도구 호출 로그와 모델 추론 로그 모두에 이 ID를 넣는다. 문제가 생기면 해당 추적 ID로 검색해서 관련 로그를 한눈에 확인할 수 있다.

2.3 성능 게이트: 지연, 토큰, 성공률 가시화하기

관측성의 목표는 결국 두 가지다. 문제가 생기기 전에 미리 감지하거나, 이미 생긴 문제를 빠르게 해결하는 것이다. 이를 위해 세 가지 성능 지표를 꾸준히 살핀다.

첫 번째 지표는 end-to-end 지연이다. 에이전트가 요청을 받고 최종 응답을 내놓기까지 걸리는 총 시간이다. 이 값이 정한 SLO를 벗어나면 알람이 울리게 한다. 지연이 늘어나는 원인은 다양하다. 모델 서버 과부하, 검색 API 응답 저하, 긴 컨텍스트에 따른 추론 시간 증가 등이다. Span 트리가 온전히 기록돼 있으면 어느 단계에서 병목이 생겼는지 금방 알 수

있다.

두 번째 지표는 토큰 소비다. 호출마다 쓴 입력 토큰과 출력 토큰의 합을 기록한다. 토큰 소비가 갑자기 급증하면 모델이 같은 내용을 반복해서 생성하고 있거나 컨텍스트에 불필요한 내용이 많이 섞여 있다는 신호다. 토큰 소비는 비용과 직결되므로 임계값을 넘는 호출은 자동으로 상세 로그를 남기도록 설정한다.

세 번째 지표는 단계별 성공률이다. 전체 호출 중 몇 퍼센트가 각 단계를 성공적으로 마쳤는지 측정한다. 특정 단계의 성공률이 갑자기 떨어지면 그 단계가 쓰는 도구나 API에 문제가 생겼을 가능성이 크다. 성공률 지표는 단순 비율에 그치지 않고 실패 유형별로 나눠 기록하면 패턴을 더 쉽게 알아챌 수 있다.

이 세 가지 기법을 갖추면 에이전트가 프로덕션에서 어떻게 동작하는지 훨씬 선명하게 파악할 수 있다. 문제가 생겼을 때 혼란 속에서 원인을 뒤지는 대신, 이미 기록된 데이터에서 원인을 읽어내는 것이 관측성 설계의 핵심이다.

3 실패 처리 아키텍처: 에이전트가 붕괴하지 않는 구조

에이전트가 실패해도 시스템 전체가 무너지지 않으려면 실패를 설계 초기 단계부터 고려해야 한다. 이 장에서는 단계를 격리해 실패가 번지지 않게 막는 방법을 다루고, 재시도와 백오프를 결정론적 코드에 맡기는 방법을 짚은 뒤, 발산을 막는 구체적 임계값 설정까지 살펴본다.

3.1 단계를 격리하고 종료 조건을 명확히 정의하기

에이전트의 각 단계를 독립적으로 다룰 수 있게 설계하면 한 단계가 실패해도 다른 단계에 영향이 번지지 않는다. 마이크로서비스 아키텍처의 circuit breaker 패턴과 같은 원리다.

예를 들어 에이전트를 검색, 결과 필터링, 응답 생성의 세 단계로 구성한다고 하자. 각 단계는 이전 단계의 출력에 의존하면서도 자체적으로 실패를 처리한다. 검색 단계에서 검색 API가 응답하지 않으면 이전 검색 결과를 캐시에서 찾아 반환하고, 캐시에도 없으면 빈 결과와 함께 상태 코드를 설정한다. 이 경우에도 필터링 단계는 정상 동작한다. 빈 입력에서도 필터링 로직이 동작하기 때문이다.

단계 격리에서 중요한 것은 각 단계의 종료 조건을 미리 명시하는 것이다. 검색 단계에는 종료 조건이 셋 있다. 검색 결과를 성공적으로 가져오거나, 캐시 히트로 이전 결과를 반환하거나, 모든 소스가 실패해 빈 결과를 반환하는 경우다. 이 세 경우 각각에 대해 후속 단계가 어떻게 동작해야 하는지를 설계 초기에 정해둔다.

종료 조건을 정의할 때 흔한 실수는 성공과 실패를 이진값으로만 나누는 것이다. 실제 프로덕션에서는 부분적 성공이라는 제3의 상태가 있다. 검색

결과 100개 중 80개만 가져오고 20개는 타임아웃되는 경우가 그렇다. 이때 완전히 실패로 처리할지, 부분 결과를 가지고 계속 진행할지 결정해야 한다. 시간이 중요한 태스크라면 대개 부분 결과로 진행하는 편이 낫고, 정확성이 중요한 태스크라면 완전한 결과를 요구하는 편이 맞다.

3.2 재시도와 백오프를 결정론적 코드에 위임하기

에이전트가 도구 호출에 실패했을 때 재시도 여부는 모델이 아니라 결정론적 코드가 결정해야 한다. 모델의 추론 budget을 귀중한 자원으로 취급한다는 뜻이자, 재시도 로직을 일관되게 유지한다는 뜻이다.

재시도 정책의 핵심은 지수 backoff다. 첫 번째 재시도는 1초 후, 두 번째는 2초 후, 세 번째는 4초 후에 시도하는 방식이다. 이렇게 하면 일시적인 네트워크 문제나 서버 과부하 상황에서 재시도 간격이 자연스럽게 늘어나며 시스템에 여유를 준다. 재시도 횟수는 보통 3회 이내로 설정하는데, 3회 넘게 재시도해도 실패하면 일시적인 문제가 아니라 지속적인 문제로 판단할 수 있다.

재시도를 구현할 때 반드시 처리해야 하는 문제가 멍등성이다. 검색 요청을 재시도할 때 이전 검색이 실제로 완료됐는지 불확실한 경우가 있는데, 이때 검색 결과가 중복으로 저장되면 안 된다. 해결 방법은 재시도 대상 작업에 고유한 idempotency key를 부여하고, 서버 측에서 이 key로 이미 처리된 작업인지 확인하는 것이다.

재시도에는 중요한 원칙이 하나 더 있다. graceful degradation, 즉 단계적 성능 저하다. 재시도 후에도 실패하면 항상 동일한 에러 메시지를 반환하는 대신 단계적으로 degraded된 응답을 시도한다. 검색 API가 완전히 실패하면 로컬에 캐시된 FAQ에서 유사 질문을 찾아 제시할 수 있다. 핵심은 어떤 상황에서도 응답을 완전히 멈추지 않는 것이다.

3.3 최대 반복 횟수와 타임아웃으로 발산 막기

에이전트가 같은 작업을 무한히 반복하며 리소스를 소진하는 현상을 발산 divergence이라고 한다. 모델이 자신의 출력을 다시 입력으로 사용하면서 판단 능력이 점점 떨어질 때 일어난다. 발산을 막으려면 두 가지 강제 장치를 설계 초기부터 내장해야 한다.

첫 번째 장치는 최대 반복 횟수다. 에이전트가 같은 목적으로 작업을 반복할 수 있는 횟수를 강제로 제한한다. 이 횟수는 에이전트의 성격에 따라 다른데, 검색과 필터링만 수행하는 에이전트라면 1회로 충분하고 복잡한 추론이 필요한 태스크라면 3회 정도가 합리적이다. 중요한 것은 이 횟수를 모델이 아니라 enforcement 메커니즘이 관리한다는 점이다. 모델이 스스로 반복을 멈출 것이라 기대해서는 안 된다.

두 번째 장치는 단계별 타임아웃이다. 전체 태스크 타임아웃과 개별 도구 호출 타임아웃을 구분해서 설정한다. 예를 들어 전체 태스크 타임아웃은 60초, 개별 검색 API 호출 타임아웃은 5초, 응답 생성 타임아웃은 10초로 잡는다. 이렇게 하면 특정 단계에서 무한 대기하는 상황을 막으면서도 전체 태스크 수준에서는 충분한 시간을 확보할 수 있다.

타임아웃과 재시도를 함께 설계할 때 주의할 점이 있다. 재시도마다 타임아웃이 리셋되면 안 된다는 것이다. 검색 API가 응답하지 않아 5초 타임아웃으로 실패하고 재시도에서 다시 5초를 기다리면 총 10초가 걸리는데, 이는 의도한 바가 아니다. 재시도 간격과 타임아웃은 별도로 관리해야 하며, 전체 소요 시간에서 타임아웃이 차지하는 비율을 합리적으로 유지해야 한다.

실패 처리 아키텍처의 목표는 에이전트가 실패하지 않는 것이 아니다. 실패는 불가피하다. 목표는 에이전트가 실패하더라도 시스템 전체가 계속 동작하는 것이다. 이 장에서 다룬 세 가지 접근, 즉 단계 격리와 재시도 위임, 발산 방지는 그 목표를 이루기 위한 실용적 수단이다.

4 운영 가능한 신뢰성: 실제로 동작하는 시스템으로 만드는 법

관측성과 실패 처리를 잘 설계하면 에이전트가 프로덕션에서 발생하는 문제 대부분을 파악하고 대응할 수 있다. 그러나 그것만으로는 부족하다. 결함을 미리 막고 고위험 행동을 통제하며 자동 복구를 체계적으로 설계해야 운영 가능한 신뢰성을 얻는다. 이 장은 이 세 가지를 다룬다.

4.1 입력 검증과 스키마 강제로 결함 사전 차단

에이전트로 들어오는 입력은 반드시 검증해야 한다. 전통적 소프트웨어 개발에서는 당연한 원칙인데, LLM 에이전트에서는 자주 빠진다. 사용자가 태스크를 설명하면 모델이 그 설명을 받아 행동을 정하는데, 입력이 불완전하거나 모호하면 그만큼 출력의 질도 떨어진다.

입력 검증을 구체적으로 어떻게 구현하는지 보자. 사용자가 에이전트에게 파일을 분석해 달라고 요청했는데 파일 경로를 주지 않았다면, 에이전트는 즉시 거부해야 한다. “파일을 분석하려면 파일 경로를 입력해주세요”라는 오류 메시지와 함께 실패를 기록한다. 모델이 알아서 적절한 파일 경로를 추측하게 되서는 안 된다. 모델의 추측에는 늘 불확실성이 끼어들고, 잘못 짚으면 이후 단계 전체가 무효가 될 수 있다.

스키마 강제는 입력의 형식과 범위를 제한하는 기법이다. 가령 에이전트가 받아들이는 날짜 형식이 “YYYY-MM-DD”라면 다른 형식은 거절한다. 숫자 범위도 마찬가지로, 페이지 번호에 음수가 들어오면 바로 거부한다. 스키마 검증은 모델보다 앞서, 즉 입력이 에이전트 시스템에 도착하는 가장 앞단에서 수행해야 한다. 그래야 불완전한 입력이 시스템 깊숙이 퍼지기 전에 막을 수 있다.

입력 검증에서 놓치기 쉬운 부분도 있다. 사용자가 준 필드뿐 아니라 필드 간의 관계도 검증해야 한다는 점이다. 시작 날짜와 종료 날짜가 둘 다 주어졌다면 시작 날짜가 종료 날짜보다 늦지 않은지도 확인해야 한다. 이런 교차 필드 검증이 불변 조건 invariant을 지켜 시스템의 무결성을 보호한다.

4.2 인간 승인 게이트로 고위험 행동 제어하기

에이전트가 파일을 삭제하거나 외부 API를 호출하거나 사용자에게 직접 메시지를 보내는 행동은 본질적으로 위험하다. 이런 고위험 행동 앞에는 반드시 사람의 승인을 거치도록 설계해야 한다. 인간 참여 Human-in-the-Loop 패턴의 핵심이 여기에 있다.

승인 게이트 자체의 구현은 복잡하지 않다. 에이전트가 삭제 작업을 수행하기 전에 작업의 대상과 범위를 사용자에게 확인받는 메시지를 띄운다. 사용자가 명시적으로 승인하면 진행하고 거부하면 취소한다. 승인과 거부는 둘 다 기록해야 한다. 시스템 logs에 “삭제 작업 승인됨” 또는 “삭제 작업 거부됨”이 남으면 나중에 검토할 때 에이전트의 행동 패턴을 분석할 수 있다.

승인 게이트의 빈도를 조절하는 게 실용적 설계의 관건이다. 모든 작업에 승인을 요구하면 에이전트의 효율이 급격히 떨어진다. 그래서 위험 등급을 정의하고 등급에 따라 승인 요구 수준을 차등 부여한다. 위험 등급을 세 단계로 나눠보자. 저위험은 읽기 전용 작업이라 승인 없이 자동으로 진행한다. 중위험은 시스템 상태를 바꾸지만 되돌릴 수 있는 작업이라 확인 메시지만 띄우고 바로 진행한다. 고위험은 되돌릴 수 없거나 외부에 영향을 미치는 작업이라 명시적 승인을 받아야만 진행한다.

중위험과 고위험의 경계는 조직과 시스템의 특성에 따라 다르다. 내부용 문서 요약 에이전트라면 파일 삭제만 고위험이고, 고객을 직접 상대하는 어시스턴트라면 모든 외부 API 호출이 고위험이 될 수 있다. 이 등급 구분은 고정된 것이 아니라 정기적으로 검토해서 조정해야 한다.

4.3 데드맨 스위치와 정지 조건으로 자동 복구 설계

예외 상황에서 에이전트를 즉시 멈출 수 있는 장치가 필요하다. 데드맨 스위치 dead man's switch가 바로 이 개념이다. 에이전트가 정해진 시간 안에 하트비트를 보내지 않으면 시스템이 자동으로 에이전트를 중지시키는 메커니즘이다.

구체적으로 어떻게 구현하는지 보자. 에이전트는 작업 중에 주기적으로 하트비트를 보낸다. 이 하트비트는 에이전트가 아직 살아있다는 신호다. 시스템은 마지막 하트비트로부터 정해진 시간 안에 다음 하트비트가 도착하지 않으면 에이전트를 강제로 멈춘다. 하트비트 간격과 타임아웃은 에이전트의 특성에 맞춰 설정한다. 빠른 응답이 필요한 에이전트라면 10초마다 하트비트를 보내고 30초 타임아웃을 둔다. 복잡한 추론이 필요한 에이전트라면 60초 간격에 180초 타임아웃이 적당하다.

정지 조건 stop condition은 데드맨 스위치와 다른 각도에서 에이전트의 행동을 제한하는 메커니즘이다. 에이전트가 해서는 안 되는 행동의 목록을 미리 정의해두고, 그 행동에 도달하면 자동으로 중지시키는 방식이다. 예컨대 “에이전트가 외부 웹사이트에 접속해 정보를 탈취하려 하면 즉시 중지”라는 정지 조건이 있다면, 에이전트가 curl 같은 도구로 외부에 요청을 보내려는 순간 시스템이 개입해 작업을 중단한다.

정지 조건은 패턴 매칭으로 구현한다. 에이전트의 도구 호출이나 인자에 특정 패턴이 나타나면 그 호출을 차단한다. 정규 표현식으로 “.rm -rf.”나 “.curl.-data.*” 같은 위험한 명령 패턴을 정의해두면, 이 패턴이 탐지될 때 도구 호출이 실행되지 않고 에이전트가 중지된다.

자동 복구를 설계할 때는 중지와 동시에 복구도 시도돼야 한다. 에이전트를 그냥 멈추기만 하면 작업이 끊기고 사용자는 아무 결과도 받지 못한다. 에이전트가 중지될 때 그때까지 진행된 작업을 저장하고, 그 상태에서 다시 시작할 수 있는 checkpoint 기능을 함께 제공한다. 그러면 에이전트가

뜻하지 않게 멈춰도 처음부터 다시 시작할 필요가 없다.

이 장에서 다룬 세 가지 기법, 즉 입력 검증과 인간 승인 게이트, 데드맨 스위치·정지 조건은 각각 다른 층위에서 에이전트의 신뢰성을 보완한다. 입력 검증은 불량 입력을 미리 막고, 인간 승인은 고위험 행동을 통제하며, 데드맨 스위치는 예외 상황에서의 무한 동작을 막는다. 이 셋을 모두 갖췄을 때 비로소 에이전트를 실제 프로덕션 환경에서 확신을 갖고 운용할 수 있다.